COMP30024 Project Part B Report

Author: Jinhao (Stephen) He 1327850, Guangheng (Jack) Tong 1129548

Introduction

Tetress is a two-player game: players place tetrominoes (each tetromino takes 4 spaces) on an 11 x 11 toroidal board one after the other until one player blocks the opponent from placing a tetromino or when the turn count reaches 150. The player loses if it's his/her turn but he/her cannot place a tetromino on the board; Or when the turn count reaches 150, the player who has more spaces on the board win the game.

This project aims to design and implement an intelligent agent that can efficiently play Tetress by taking strategic actions. In developing the agent, several artificial intelligence techniques were explored. The goal was to maximize the agent's winning chance. This report provides a comprehensive overview of the agent's design, performance evaluation, and the creative approaches used to improve gameplay.

Agent Strategy and Design

Our Tetress agent employs the minimax search algorithm to evaluate and select optimal actions. This choice was made due to the efficiency and suitability of the minimax algorithm in adversarial games.

Search Algorithm: Minimax and alpha-beta pruning

- Minimax: the minimax algorithm recursively explores potential game states, aiming to maximize the agent's score while minimizing the opponent's potential gains. It assumes both players play optimally, helping the agent prepare for worst-case scenarios.
- Alpha-Beta Pruning: to optimize minimax, alpha-beta pruning was implemented to skip branches that wouldn't influence the final decision, significantly reducing computation time.

Modifications and Enhancements

Our agent employs a strategic game-playing approach using lookahead search and action evaluation to identify the optimal actions:

- Lookahead Search: The 'evaluation_board_with_lookahead' function forms the core of our search strategy. By recursively evaluating future game states up to a given depth, it identifies actions that minimize the opponent's opportunities. This function tracks potential actions through 'ActionNode' instances to explore deeper layers of possible actions, ensuring a comprehensive evaluation.
- Evaluation and Action Selection: The 'evaluate_and_select' function complements this lookahead search by selecting actions that minimize the opponent's opportunities. It compares the number of possible opponent actions after simulating each move and then chooses the move that most restricts the opponent's future actions.

Our agent dynamically adapts its strategy based on game state and opponent actions:

- Initial Actions: Each player starts with a specific hardcoded initial action. For example, the RED player (the first player)'s first move is pre-determined to place a tetromino in a strategic central location on the board. After this initial action, the agent transitions to an adaptive strategy.
- Dynamic Game Play: Once the game progresses beyond the initial move:

    - Our agent evaluates moves using 'evalution_board_with_lookahead' to anticipate future states up to a specified depth.
    - Using a combination of lookahead and minimizing the opponent's possible actions, the agent chooses the action that provides the greatest advantage while limiting the opponent's opportunities.

This approach combines sophisticated search strategies, iterative deepening, and strategic evaluation to create a highly competitive agent capable of maintaining a strong board position and reducing the opponent's chances.

Evaluation Function

Our agent leverages robust action generation and evaluation functions to identify strategically advantageous actions:

- Action Generation: The 'generate_possible_actions' function computes all valid placements by:
    - Identifying coordinates directly adjacent to existing pieces of the same colour using 'find_valid_adjacent_coords' function.
    - Recursively expanding possible placements by simulating successive adjacent placements.
- Simulating and Evaluating Actions: The 'simulate_action' function applies a potential action to a simulated game state, accounting for any filled rows or columns through 'eliminate_filled_row_or_column'. This simulation enables accurate assessment of an action's impact.

Performance Evaluation

Testing Methodology

To assess the agent's performance, we conducted many experiments against self-implemented random agent 'agent_random' and greedy agent 'agent_greedy', using automatic testing enabled by 'run_games.sh' file. Each test match was conducted with the agent playing both as RED and BLUE for an equal number of games to account for potential first-move advantages.

Results

Due to the space limit, the results of agents playing against each other have not been placed on a table. However, the general result is that the agent with a deeper depth outperforms the agent with a shallower depth, until a certain depth where after that the performance diminishes.

The below list some general results:

Agent (depth = 3) as RED vs Random Agent as BLUE --- Agent winning rate = 32 wins/40 tests = 80%

Agent (depth = 3) as BLUE vs Random Agent as RED --- Agent winning rate = 34 wins/40 tests = 82.5%

Agent (depth = 3) as RED vs Greedy Agent as BLUE --- Agent winning rate = 40 wins/40 tests = 100%

Agent (depth = 3) as BLUE vs Greedy Agent as RED --- Agent winning rate = 34 wins/40 tests = 85%

Note:

Greedy Agent is designed as an agent intends to maximize the number of its next actions without considering what the opponent's next action will be.

Insights

- Lookahead Depth: The lookahead search significantly improves performance against random opponents, allowing the agent to anticipate future threats. The deeper the search depth, the earlier the agent can prevent future threats; However, this effect can only work until a certain depth, then the effect of lookahead starts to diminish. This might be due to the smaller probabilities of which a future threat might happen.
- Opponent Action Minimization: The strategy of minimizing opponent actions consistently yielded favourable actions, allowing the agent to adjust its strategy to account for different opponents.
- Time Constraints: certain challenging decisions (especially in the middle of the game, where many possible actions are presented) took longer than desired, highlighting areas where additional pruning could enhance efficiency. This area could be the focus of future improvements, considering additional heuristics information, such as pieces coverage and connectivity.
- There is **no one-size-fits-all solution**: The agent could have a high winning rate against greedy agents when the move is more predictable than a random agent. No agent can assure a 100% winning rate in all situations. **Exploiting the opponent's strategy** is a shortcut to win.

Technical and Creative Aspects

Optimizations

- Lookahead Search Depth Optimization:
    - Problem: ensuring an optimal lookahead depth within the computational constraints (180s, 250MB) was challenging, especially with the large branching factor of the game.
    - Solution: Implementing a depth limit based on iterative deepening search allowed the agent to explore until a certain depth while maintaining response speed. Based on numerous experiments, we set the **RED depth limit** = 3, and the **BLUE depth limit** = 3. We also set a **branching factor threshold** = 3, which means we won't exploit more than 3 equivalent best action nodes. They yield the best performance against random agents, >85% and >80% respectively.
- Action Evaluation Caching:
    - Problem: Evaluating similar board states repeatedly during the game resulted in redundant calculations.
    - Solution: We attempted to use a caching mechanism to store the evaluation of common specific game states. This approach minimized re-computation, reducing the evaluation time for previously assessed states. (However, due to the limit time for the assignment, this approach has not been successfully implemented yet.)
- Evaluation Function Refinement:
    - Motivation: In Tetress, reducing the opponent's strategic flexibility is crucial.
    - Implementation: The agent's evaluation function prioritizes moves that leave the opponent with the fewest legal options, effectively restricting their ability to continue the game.
- Randomized Action Selection:
    - Implementation: in cases where multiple equivalent actions are identified, the agent selects randomly among them.

    - Purpose: this prevents predictability and reduces the likelihood of an opponent exploiting known patterns.
- Opponent-Aware Depth:
    - Implementation: The agent dynamically adjusts the depth of its lookahead strategy based on the opponent's perceived capabilities and their recent moves. (To-be-implemented.)
    - Effectiveness: This ensures that computation resources are used efficiently, avoiding deep searches where they aren't needed.

Data Structures

- ActionNode for Move Tracking:
    - Implementation: The 'ActionNode' class serves as a lightweight, recursive structure to trace move sequences from any given depth back to the initial action.
    - Advantages: This structure simplifies lookahead evaluation, enabling efficient backtracking while avoiding repeated calculations for previously evaluated states.
- Coordinate Set and Tuples:
    - Implementation: The set-based approach to track coordinates ensured that generated actions were unique and quickly accessible.
    - Advantages: Using sets and tuples minimize unnecessary memory usage, allows rapid checks for valid moves, and ensures a fast lookup of occupied positions.

Independent Research

To create a strong agent, independent research was conducted in the following areas

- Monte Carlo Tree Search (MCTS):
    - Rationale: MCTS is widely used in game-playing algorithms due to its ability to balance exploration and exploitation.
    - Analysis: Although due to its complexity MCTS is not directly implemented in our agent, research into MCTS provided valuable insights into how simulations and random sampling can aid strategic decision-making. Understanding MCTS principles influenced our approach to lookahead and dynamic depth adjustment (to-be-implemented).
- Resource-Constrained Adversarial Search:
    - Rationale: Tetress has limited time and space limit.
    - Research Application: Research into adversarial search optimizations, including iterative deepening and alpha-beta pruning, guided our design.
- Game Theory:
    - Rationale: Game theory provides foundational principles for competitive decision-making.
    - Research Insights: Reviewing game theory principles, including the Nash equilibrium, influenced the strategic approach to action evaluation. The agent's emphasis on minimizing opponent actions aligns closely with minimax logic.

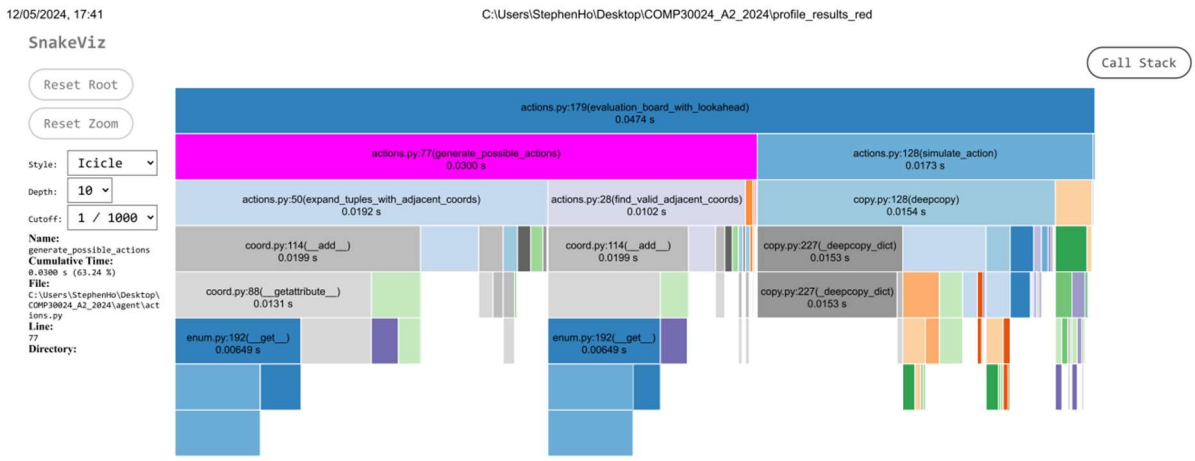
This independent research provided essential insights and innovative ideas that shaped the agent's strategy and implementation.

Supporting Work

In addition to researching and implementing core strategies, several supplementary tools and methodologies were employed for the development, testing, and refinement of the agent.

Tools and Scripts

- Development and Testing: Code Versioning and Collaboration: GitHub Repository: A private repository was used for version control, enabling smooth collaboration and regular backups.
- Benchmark Agent Development: A **simple random** agent 'agent_random' was developed that consistently makes random moves without considering future repercussions. This served as a baseline opponent to assess the agent's strategic decision-making. Besides, a simple **bash** script is designed to run the game automatically 20 (or any given number of) times.
- SnakeViz: For optimizing our agent, particularly in terms of computational efficiency and response time, we employed SnakeViz, a browser-based graphical visualization tool for Python profiling. It helped us to identify performance bottlenecks and verify improvement. By leveraging SnakeViz, we not only enhanced the agent's performance but also gained deeper insights into Python's execution flow and resource usage.



Conclusion

This project aims to develop an agent for playing the Tetress game, utilizing adversarial search and efficient evaluation functions. Our agent demonstrated considerable competitive skill against a range of opponents, showcasing the important of lookahead strategies, heuristic evaluation, and opponent-aware minimization techniques.

Key accomplishments included the following:

- Sophisticated Game-Playing Approach: the agent utilized iterative deepening and adversarial search, balancing exploration with strategic exploitation to minimize opponent actions while maximizing our potential actions.
- Adaptive Strategy: the agent dynamically adjusted its strategy based on the game context and the opponent's tactics, ensuring adaptability across various game scenarios.
- Optimization and Creative Techniques: Using tools like SnakeViz allowed us to iteratively refine and optimize the agent's computational efficiency and strategic decision-making.

Areas for further exploration and improvement:

- Advanced Evaluation Heuristics: there is still room for more sophisticated heuristic functions that could better evaluate potential board states.

- Improved Opponent Modelling: a deeper understanding of opponent tendencies could lead to more tailored strategies that exploit opponents' weaknesses.

In summary, this project combined algorithmic principles, strategic insights, and profiling tools to create a robust agent for playing Tetress. The agent is competitive against a range of opponents and provides a learning opportunity for future work in developing even more sophisticated adversarial game-playing agents.